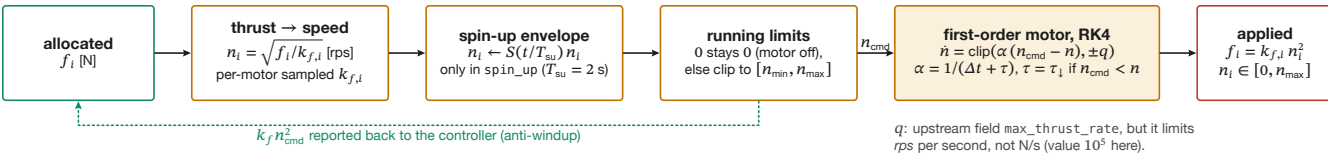
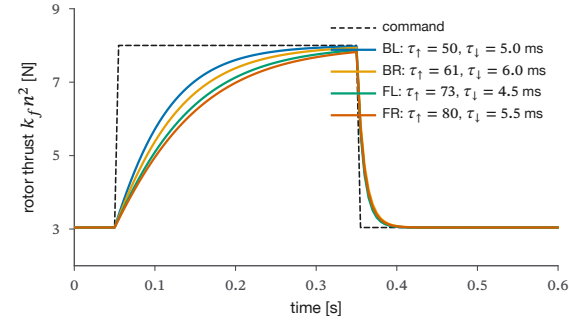


a From allocated thrust to applied force, every physics step runtime/loop.py, sim/actuation.py, native Thruster integrators

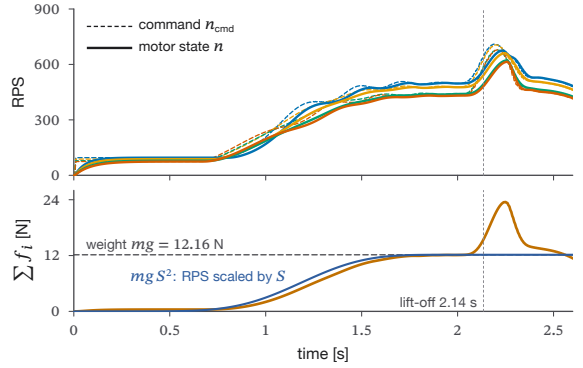


b Asymmetric lag: slow up, fast down



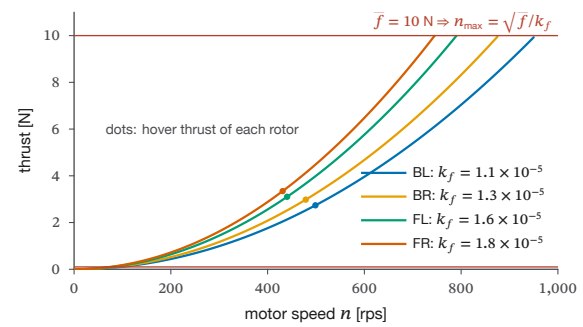
Transcribed native integrator (sim/native_thruster.py) driven through NativeRpsActuator, $\Delta t = 5$ ms, RK4, discrete mixing. Command steps from hover to 8 N at 0.05 s and back at 0.35 s. Values are the synthetic plant's deliberately unequal motors, not ARL measurements.

c Spin-up on the ground (synthetic helix run)



The first positive command jumps to n_{min} (running minimum, 75–95 rps), so the clamped target is not C^4 even though S is. The motor state still starts from 0 and rises with τ_1 . The vehicle lifts off when $\sum f_i$ exceeds mg and the takeoff reference starts at $t = T_{su}$.

d $f = k_f n^2$ per motor, up to n_{max}



Units

n is in rps (revolutions per second): $\text{rpm} = 60 n$, $\omega_r = 2\pi n$ [rad/s]. If written as $f = k_\omega \omega_r^2$, then $k_\omega = k_f / (2\pi)^2$: reusing k_f with rad/s would be off by $(2\pi)^2 \approx 39.5$.

Same thrust, different speed

Each motor keeps its own sampled $k_{f,i}$ (reset-time random draw in Isaac; fixed unequal values in the synthetic plant), so equal thrust needs unequal rps; logs record both commands and states.

Not modelled here

No CFD, ESC current/voltage, back-EMF or rotor inertia; the battery model only constrains the thrust command and is off by default.